

PAPER

Lifecycle-Aware State Spilling (LASS) for Mitigating the Latency Cliff in Managed Blockchain Clients

Yeochan YOON^{†a)}, *Nonmember* and Yangmo YOO^{†b)}, *Member*

SUMMARY Predictable execution latency is critical for blockchain validators to meet consensus deadlines. However, managed blockchain clients often suffer from severe execution jitter caused by the interaction between complex in-memory state management and reactive garbage collection (GC). This paper identifies a performance instability termed the Latency Cliff, where intensive object allocation during state-trie updates overwhelms the runtime's reclamation capacity, leading to large Stop-The-World (STW) pauses and amplified tail latency. To mitigate this issue, we propose Lifecycle-Aware State Spilling (LASS), a lightweight mechanism that proactively manages the lifecycle of ephemeral state objects. By monitoring runtime memory signals, LASS preemptively spills transient state data to persistent storage, preventing the accumulation of short-lived objects that trigger massive GC pauses. We evaluate LASS on two enterprise clients: JVM-based Hyperledger Besu and CLR-based Nethermind. Results show LASS reduces total GC time by 96.2% and decreases P99 STW pause durations by 35.4% in Besu. In Nethermind, proactive stabilization of the Large Object Heap (LOH) improves sustained transaction throughput by 5.7% while reducing tail latency. These findings demonstrate that proactive lifecycle management substantially improves predictability in managed blockchain clients without modifying the underlying runtimes.

key words: Blockchain, Managed Runtime, Garbage Collection, Tail Latency, State Management

1. Introduction: The Predictability Challenge in Managed Clients

The stability and performance of validator nodes in decentralized networks depend not only on deterministic state transitions for reproducibility, but also on highly predictable execution latency. In practice, however, managed blockchain clients frequently exhibit non-deterministic execution jitter. Standard garbage collection (GC) cycles introduce unpredictable pauses that can severely disrupt time-sensitive Byzantine Fault Tolerance (BFT) consensus rounds. These pauses may degrade network health by causing missed slots or accidental forks.

The root of this predictability challenge lies in the interaction between cryptographic state-trie management and reactive GC heuristics. To characterize this systemic instability, we formalize the concept of the Latency Cliff (C_L). The Latency Cliff refers to the

critical threshold at which the object allocation rate (R_a) required for materializing the world state exceeds the concurrent reclamation rate (R_r) of the underlying garbage collector. When this threshold is crossed, promotion pressure and object graph complexity overwhelm reactive reclamation mechanisms, resulting in severe performance degradation.

To address this fundamental disconnect, this paper introduces Lifecycle-Aware State Spilling (LASS). LASS acts as a lightweight, lifecycle-aware control layer that explicitly decouples storage persistence from the unpredictable GC lifecycle. This paper makes the following contributions:

- **Problem Formalization (Latency Cliff).** We identify and formalize the Latency Cliff, demonstrating how extreme allocation rates ($R_a > R_r$) in state-trie structures can destabilize reactive garbage collectors.
- **Proactive Architecture (LASS).** We design and implement LASS, a runtime-agnostic control layer that intercepts and manages the lifecycle of ephemeral state objects.
- **Cross-Runtime Evaluation (JVM & CLR).** We conduct a comprehensive evaluation across two enterprise-grade blockchain clients.

2. Background and Related Work

2.1 Background: State-Trie Lifecycle and Memory Pressure

The Ethereum state is anchored by a Modified Merkle Patricia Trie (MPT) [1], a persistent authenticated data structure used to maintain account states. The MPT consists of three primary node types: Branch Nodes (17-entry arrays), Extension Nodes (shared prefix paths), and Leaf Nodes (terminal path segments containing state data). This structure introduces specific memory management challenges in blockchain clients implemented on managed runtimes.

During block execution, updates to account state trigger modifications of leaf nodes in the trie. Because the MPT is hash-authenticated, each modification requires recomputing the hashes along the entire path from the modified leaf node to the root. As a result, a single state update generates multiple transient node

[†]The authors are with the Department of Electronic Engineering, Sogang University, Seoul, Korea.

a) E-mail: ycyoon@sogang.ac.kr

b) E-mail: ymyoo@sogang.ac.kr

DOI: 10.1587/trans.E0.???.1

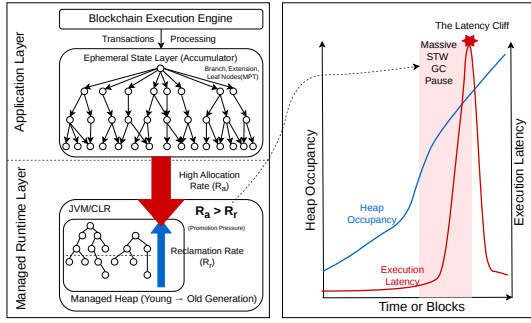


Fig. 1 Mechanism of the latency cliff in managed blockchain clients.

objects. When processing an entire block containing thousands of transactions, this mechanism generates a large volume of short-lived objects that rapidly accumulate in the managed heap.

Although many of these objects are short-lived, some survive minor garbage collection cycles and become promoted to older heap generations. Once promoted, they form large and complex object graphs that significantly increase the cost of garbage collection marking. Consequently, the memory management overhead grows nonlinearly as execution proceeds.

Modern Ethereum clients employ several optimizations to mitigate storage overhead. For example, Bonsai-style storage reduces persistent storage costs by maintaining location-based state representations. However, blockchain clients must still support chain reorganization, which requires maintaining intermediate execution states. To handle this, clients typically implement Layered States combined with in-memory Accumulators, where the accumulator acts as a transient diff layer buffering state updates before committing them to the underlying trie.

While these mechanisms improve storage efficiency, they also introduce additional long-lived in-memory state layers. These layers significantly increase promotion pressure in the managed heap, causing more objects to survive garbage collection cycles and move into older generations.

The interaction between transient trie updates and long-lived in-memory state layers creates a characteristic latency instability in managed blockchain clients, termed the **Latency Cliff** (Fig. 1). As illustrated on the left side of Fig. 1, the execution engine continuously materializes ephemeral state objects that form highly connected graphs. When the allocation rate of these objects (R_a) exceeds the garbage collector's reclamation rate (R_r), sustained promotion pressure builds within the managed heap. Consequently, as depicted on the right, this steady heap accumulation eventually breaches a critical threshold, forcing a massive Stop-The-World (STW) garbage collection. This abrupt transition from stable execution to severe latency spikes characterizes the Latency Cliff.

2.2 Related Work: Blockchain State Management and Storage Optimization

Since the Ethereum state is anchored by complex trie structures, several studies have attempted to optimize the Authenticated Data Structure (ADS) or storage engine to reduce system overhead. For example, Ponnapalli et al. proposed the Distributed Merkle Patricia Tree (DMPT) [2], which vertically shards the tree across the memory of multiple nodes to eliminate I/O bottlenecks and enable parallel processing. Other approaches, such as GEM²-tree [3], propose optimized multi-level index structures designed specifically to reduce gas costs and accelerate authenticated queries. However, while techniques like DMPT improve parallelism, they fundamentally weaken network decentralization, as a single node can no longer independently complete verification without relying on others. Beyond structure, to mitigate expensive disk operations, BLock-LSM [4] co-designs Ethereum's data characteristics with a Log-Structured Merge (LSM) tree, aiming to mitigate write amplification during data synchronization. Similarly, the DSM-TREE architecture in RAIN-BLOCK [5] introduces I/O-Helpers that prefetch data and proofs from storage nodes, thereby entirely removing disk I/O from the critical path of transaction processing. While these existing works successfully optimize storage layouts, physical I/O latency, or authenticated data structures, they largely treat the managed runtime of the client as a black box. Existing solutions focus on optimizing data after it leaves the application layer, completely overlooking the fundamental conflict between in-memory cryptographic state-trie management and reactive garbage collection (GC) heuristics.

2.3 Related Work: Distributed Storage and Data Reduction Techniques

To alleviate the massive storage burden on individual nodes, existing efforts rely on data distribution, erasure coding, and partial storage. Data distribution techniques, such as Elastico [6] and OmniLedger [7], partition the mining network into smaller committees so that each node only maintains a subset of the global state. Alternatively, frameworks employing Network Coded (NC) distributed storage leverage erasure coding to dramatically reduce storage requirements [8]. Furthermore, partial storage approaches like Jidar (Jigsaw-like Data Reduction) [9] allow nodes to selectively store only the transactions and Merkle branches they are interested in. While these methods effectively reduce the physical disk footprint, they often incur substantial computational costs during state reconstruction or partially compromise strict network decentralization due to localized trust assumptions.

2.4 Related Work: Garbage Collection and Memory Management

Garbage collection has long been recognized as a major source of latency instability in managed runtime systems. Modern collectors attempt to mitigate pause time through various strategies. For example, the Garbage-First (G1) collector prioritizes heap regions with the highest reclaimable memory to reduce pause duration and improve predictability [10]. Other approaches, such as the continuously concurrent compacting collector (C4), attempt to eliminate long pause times by performing most compaction work concurrently with application execution [11]. Similarly, the .NET runtime employs a generational garbage collector that separates allocations into different heaps, including the Large Object Heap (LOH), which introduces unique fragmentation and reclamation behavior for large objects [12]. Prior work has also explored scalable memory allocation strategies, such as Hoard [13], which aim to mitigate contention and memory fragmentation in multithreaded environments.

Recent efforts in blockchain and other data-intensive domains have recognized the detrimental impact of garbage collection on blockchain consensus. Bläser et al. [14] introduced a custom, fully incremental garbage collector specifically designed for WebAssembly (Wasm)-based blockchains to eliminate unbounded Stop-The-World (STW) pauses. However, this top-down solution necessitates building a completely new garbage collector from scratch and is tightly coupled to specific runtime environments and languages (e.g., Motoko).

In other data-intensive domains such as in-memory analytics, several approaches attempt to reduce GC pressure by bypassing the managed heap or modifying execution behavior. For example, Facade [15] provides a compiler and runtime that move large objects off-heap to avoid excessive GC overhead, while Yak [16] redesigns the garbage collector to better support big-data workloads. Other approaches, such as the WASP scheduler, jointly optimize task granularity and parallelism by predicting memory usage to avoid exceeding heap capacity [17]. Additionally, Jang et al. [18] proposed an eager spill mechanism (e-spill) for Apache Spark, which employs a slow-start model that dynamically identifies an optimal spill threshold by monitoring the ratio of spill time to GC time during execution.

Our proposed LASS shares the fundamental philosophy of utilizing preemptive data spilling to bypass reactive GC heuristics. However, the operational contexts differ significantly. While e-spill relies on dynamic feedback loops based on observed execution times to optimize large-scale data analytics, LASS must operate under the strict deterministic requirements of Byzantine Fault Tolerance (BFT) consensus. There-

fore, LASS introduces a statically configured, protocol-aware control plane specifically tailored to the lifecycle of cryptographic state-trie structures in blockchain clients.

3. Design of Lifecycle-Aware State Spilling (LASS)

The key insight behind LASS is that the Latency Cliff is fundamentally caused by the uncontrolled promotion of ephemeral state objects in managed runtimes.

3.1 Design Overview

Managed blockchain clients execute blocks by materializing a large mutable working set to represent pre-commit state transitions. We define this working set as the ephemeral state layer, which comprises:

- In-memory accumulator structures that buffer state updates prior to commitment.
- Layered state views maintained to support chain reorganizations and fork choice.
- Transient trie nodes created during path re-materialization and re-hashing.

In the Besu implementation, this layer primarily corresponds to the in-memory Bonsai world state accumulator and the intermediate trie node objects created during state updates through the WorldStateUpdater. LASS intervenes at the point where buffered state diffs are materialized for persistence, allowing the governor to serialize and evict these objects before they accumulate in the Old Generation. Currently, the physical lifetime of this ephemeral state is implicitly governed by the garbage collector, rendering the reclamation of highly connected object graphs reactive and entirely decoupled from the actual protocol lifecycle. Lifecycle-Aware State Spilling (LASS) shifts this paradigm from a reactive model—which persists data only upon resource exhaustion—to a proactive model by introducing an explicit control plane. By explicitly controlling when cryptographic state leaves the managed heap, LASS aligns the lifetime of these complex object graphs with application-governed protocol boundaries rather than arbitrary memory pressure.

3.2 Architectural Design and Placement

LASS is implemented as a lightweight, runtime-agnostic governor situated immediately above the persistent storage backend and below the execution engine. The governor strictly controls the lifecycle of the ephemeral state layer without intercepting arbitrary allocations or modifying the underlying garbage collector. Specifically, LASS enforces an early materialization and eviction policy through an operation termed state spilling. This involves:

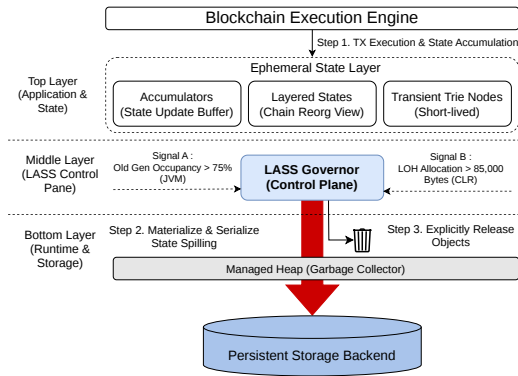


Fig. 2 Architectural placement and operational flow of the LASS governor within a managed blockchain client.

- Materializing the active ephemeral representation (e.g., accumulator layers and buffered trie diffs) into a serialized form suitable for the client’s native persistent backend.
- Releasing the corresponding managed objects immediately after the serialization process completes.

To achieve this, the LASS governor requires only minimal client-level abstractions: an explicit handle to the currently active ephemeral state layer, a deterministic serialization path, and a safe reset mechanism for the in-memory representation post-materialization.

Importantly, LASS is fundamentally distinct from traditional garbage collection tuning. While GC tuning reshapes collector heuristics without altering the underlying object graph, LASS directly eliminates the primary source of marking complexity and promotion pressure by aggressively advancing the eviction of long-lived ephemeral state objects.

Fig. 2 illustrates the architectural placement and operational flow of LASS within a managed blockchain client. During normal execution, the transaction execution engine generates updates to the ephemeral state layer, which temporarily accumulates state diffs and trie node objects. Instead of allowing these objects to remain in the managed heap until garbage collection is triggered, the LASS governor continuously monitors runtime memory signals. When predefined thresholds are reached, the governor proactively materializes the buffered state into the persistent storage backend through controlled state spilling. Once serialization completes, the corresponding in-memory objects are immediately released, preventing excessive promotion into long-lived heap regions. By inserting this lightweight control plane between the execution engine and the storage backend, LASS effectively decouples state persistence from the reactive behavior of the garbage collector.

3.3 Runtime-Specific Control Signals

Because LASS does not require runtime modifications

or GC instrumentation, it relies on runtime-specific observability signals to anticipate and prevent the Latency Cliff.

JVM-Based Client (Hyperledger Besu) For the JVM runtime, the governor monitors Old Generation occupancy via standard management interfaces to proactively mitigate promotion pressure before the G1 collector schedules mixed or full collections. A spill is triggered when:

- Old Generation occupancy exceeds 75%.
- This threshold is consistently breached across three consecutive samples collected at 100 ms intervals. Rather than synchronizing with GC phases, this policy preemptively reduces the volume and connectivity of long-lived state objects destined for the Old Generation.

CLR-Based Client (Nethermind) In contrast, the CLR runtime governor focuses on Large Object Heap (LOH) stability, as LOH fragmentation is primarily driven by large allocation patterns rather than generational promotion dynamics. A spill is triggered immediately when:

- An allocation request for a serialization buffer exceeding 85,000 bytes is observed while an active ephemeral state layer is present. This threshold explicitly aligns with the LOH allocation boundary, effectively preventing the accumulation of large, short-lived buffers that induce blocking Generation 2 collections.

Although the concrete trigger signals differ across runtimes, the core objective of the LASS governor remains identical. In both environments, the governor proactively limits the lifetime of ephemeral state objects before they accumulate in long-lived heap regions. The JVM implementation monitors Old Generation occupancy to prevent excessive promotion pressure, whereas the CLR implementation leverages the Large Object Heap (LOH) allocation boundary as an early indicator of large transient buffer creation. These runtime-specific signals therefore represent different observability points for enforcing the same architectural principle: proactively controlling the lifecycle of ephemeral state objects to prevent the Latency Cliff.

4. Evaluation

4.1 Experimental Setup

To rigorously evaluate the impact of Lifecycle-Aware State Spilling (LASS) on execution predictability and system throughput, we conducted experiments on a dedicated high-performance testbed. Our evaluation aims to isolate the impact of garbage collection behavior under controlled and reproducible workloads while

maintaining realistic blockchain client execution conditions.

Hardware and Software Environment All experiments were conducted on a server equipped with dual-socket Intel(R) Xeon(R) Gold 5317 CPUs @ 3.00GHz, providing a total of 24 physical cores (48 logical threads). The system was provisioned with 512 GB of RAM to accommodate large in-memory state-trie structures generated during block execution. To minimize external storage bottlenecks and ensure that measured overheads reflect the architectural tradeoffs of LASS rather than disk latency, all persistent storage backends were hosted on high-speed 450 GB NVMe SSDs. The host operating system was Rocky Linux 8.10 (Kernel 4.18.0). For cross-runtime evaluation, we used two widely deployed managed blockchain clients. The JVM-based experiments were conducted using Hyperledger Besu v24.1.1 running on OpenJDK 17.0.13 with the G1 garbage collector enabled. The CLR-based experiments were conducted using Nethermind v1.37.0 running on the .NET 10.0 runtime. Both clients were configured as block-producing validator nodes in isolated development networks. Although the experiments were conducted on a high-memory server to reduce external resource constraints, the Latency Cliff phenomenon is fundamentally driven by the imbalance between object allocation and reclamation rates within managed runtimes. Therefore, similar GC-induced latency instability may also occur on smaller validator nodes when workloads generate sufficient state-trie mutation pressure.

Transaction Workload We evaluate LASS using a synthetic, single-type workload rather than mainnet or testnet trace replay. Each transaction invokes a state-bloating function `bloat(startIndex, count)` in a dedicated Solidity contract (StateBloater). This function writes to count distinct storage slots in the contract’s storage space, which triggers corresponding updates to the underlying Merkle Patricia Trie (MPT). As a result, each transaction induces both persistent storage writes and the creation of multiple transient trie nodes within the client. The workload generator runs multiple worker threads that continuously submit transactions without rate limiting, allowing the system to operate at its maximum sustainable throughput. The parameters of the workload are fixed for each experimental run in order to maintain consistent allocation pressure across configurations. In the Besu experiments, the driver uses 20 worker threads and performs 200 storage-slot updates per transaction. In the Nethermind experiments, the driver uses 14 worker threads and performs 300 storage-slot updates per transaction. Each experimental run lasts 600 seconds. This workload is intentionally designed to generate controlled and repeatable allocation pressure within the managed run-

time. By using a single transaction type with deterministic state updates, we ensure that any observed differences in latency behavior, garbage collection overhead, or throughput between the Baseline and LASS configurations are attributable to the LASS mechanism itself rather than to variations in transaction mix. While this synthetic workload does not represent a realistic production transaction distribution, it provides a controlled environment for isolating GC-induced latency behavior. Evaluating LASS under heterogeneous workloads or mainnet trace replay remains an important direction for future work.

Block Production and Transactions per Block

Execution in Ethereum clients is fundamentally block-based. In our experimental setup, the workload generator continuously submits transactions while the validator node produces and seals blocks according to the configured block interval. In the development networks used for our evaluation, the block interval is configured to approximately one second. Under sustained transaction submission, the number of transactions included in each block therefore depends primarily on the achievable throughput of the client. As a result, the mean number of transactions per block approximately corresponds to the sustained throughput measured in transactions per second. In the reported experiments, Besu processes approximately 109 transactions per second in the baseline configuration and approximately 114 transactions per second with LASS enabled, resulting in roughly 109–114 transactions per block. Similarly, Nethermind processes approximately 198 transactions per second in the baseline configuration and approximately 209 transactions per second with LASS, resulting in roughly 198–209 transactions per block. All latency metrics reported in this work correspond to transaction execution latency observed during client-side block processing.

Implementation of LASS LASS is implemented with minimal modifications to the client codebases. In Besu, we introduce two new components: a `MemoryPressureMonitor` and a `TransactionSpillStorage` module, totaling approximately 330 lines of new code. The `MemoryPressureMonitor` monitors JVM garbage collection overhead in real time and triggers transaction spilling when memory pressure exceeds configured thresholds. `TransactionSpillStorage` provides a RocksDB-backed storage layer that temporarily offloads pending transactions from memory. LASS is integrated at the transaction pool layer (e.g., `PendingTransactions / AbstractPrioritizedTransactions`), where pending transactions can be selectively spilled and later restored without modifying the core execution pipeline or the `WorldStateUpdater` logic. For Nethermind, we do not introduce a separate state-spill module. Instead, we leverage the client’s existing garbage collection control mechanisms

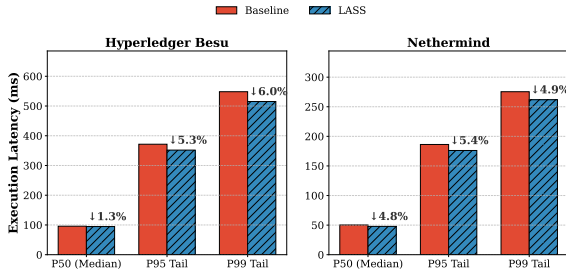


Fig. 3 Execution tail latency comparison between the baseline and LASS configurations.

implemented in the Merge Plugin GC and GCScheduler modules (approximately 470 lines of code). These components suppress garbage collection during block processing using NoGCRegion and perform scheduled garbage collection after block completion with optional large-object-heap (LOH) compaction. This configuration stabilizes execution latency under high allocation pressure while requiring no modification to the World-State or transaction execution pipeline.

4.2 Mitigating Execution Tail Latency

The primary objective of LASS is to prevent the unpredictable execution delays caused by massive STW pauses. Fig. 3 illustrates the execution latency distribution (P50, P95, and P99) for both the Baseline and LASS configurations.

As shown in the results, LASS maintains or slightly improves the median execution time (P50), which remains stable for both Besu (95.90 ms to 94.67 ms) and Nethermind (50.29 ms to 47.88 ms). However, the critical improvement is observed at the tail end of the distribution. By circumventing the Latency Cliff, LASS successfully reduced the P99 tail latency in Besu from 548.08 ms to 515.20 ms (a 6.0% reduction). Similarly, in Nethermind, the P99 latency decreased from 275.41 ms to 261.88 ms (a 4.9% reduction). This result indicates that proactively clearing the ephemeral state layer stabilizes block processing latency and mitigates tail latency spikes. It is worth noting that the reduction in tail latency appears more modest than the reduction in total GC time reported later in Section 4.3. This discrepancy arises because garbage collection pauses constitute only a portion of the total transaction execution time. Block execution also includes transaction validation, trie updates, cryptographic hashing, and storage I/O, which together dominate the overall execution cost. Consequently, eliminating GC pressure primarily stabilizes latency outliers rather than drastically reducing the absolute execution time of typical transactions. The main benefit of LASS is therefore the suppression of extreme latency spikes caused by major garbage collection events, which improves execution predictability without significantly altering the baseline execution cost of transaction processing.

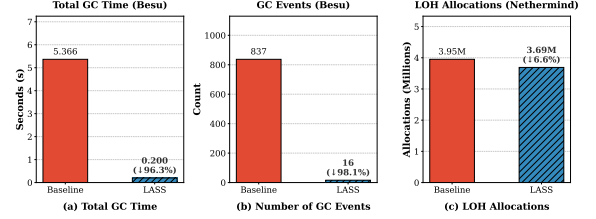


Fig. 4 Reduction of Garbage Collection (GC) overhead in Hyperledger Besu before and after applying LASS. (a) Total GC time in Hyperledger Besu (JVM). (b) The number of major GC events in Besu. (c) The number of Large Object Heap (LOH) allocations in Nethermind (CLR).

4.3 JVM & CLR GC Behavior

The root cause of the Latency Cliff is the excessive promotion pressure that overwhelms the managed runtime’s garbage collector. Fig. 4 illustrates the reduction in GC overhead and memory pressure achieved by LASS across two different runtime environments; the JVM-based Hyperledger Besu and the CLR-based Nethermind.

By proactively materializing and releasing ephemeral state objects before they accumulate in the Old Generation, LASS substantially reduces generational promotion pressure. In the JVM-based Besu client, this mechanism ensures that the garbage collector rarely needs to perform expensive major collection cycles. As shown in Figure 4(a), the total time spent on garbage collection decreased by 96.2%, from 5.366 seconds to 0.200 seconds. This is directly supported by the data in Figure 4(b), which shows that the number of major GC events dropped dramatically from 837 in the Baseline to only 16 under LASS. Additionally, the P99 Stop-The-World (STW) pause duration was reduced by 35.4%, decreasing from 47.70 ms to 30.82 ms. A consistent stabilization effect was observed in the CLR-based Nethermind client. In the .NET runtime, excessive state objects often trigger frequent collections in the Large Object Heap (LOH), which is particularly disruptive to execution stability. As illustrated in Figure 4(c), the number of LOH allocations decreased from approximately 3.95M to 3.69M. This indicates that LASS’s proactive state spilling effectively suppresses large-object allocation bursts by decoupling state persistence from the managed heap. These combined results demonstrate that LASS consistently alleviates memory pressure and mitigates the Latency Cliff across fundamentally different managed runtimes.

4.4 Overall Throughput Improvement

A potential concern with the LASS architecture is whether the I/O overhead of explicitly materializing and serializing state objects to the persistent storage (Step 2 in Fig. 2) outweighs the benefits of bypass-

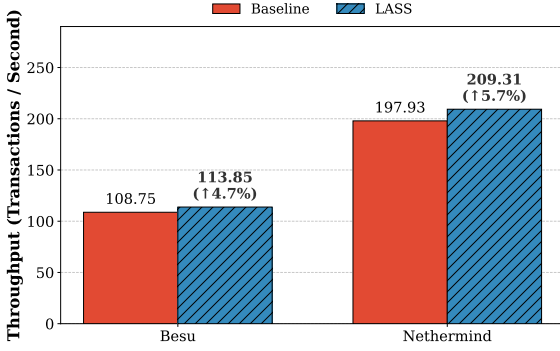


Fig. 5 Overall transaction throughput (TPS) improvement.

ing the GC. As illustrated in Fig. 5, the performance benefits of bypassing the GC completely offset the additional I/O serialization overhead introduced by the LASS governor. In the JVM-based Besu client, LASS increased the sustained throughput from 108.75 TPS to 113.85 TPS. Similarly, in the CLR-based Nethermind client, the throughput improved from 197.93 TPS to 209.31 TPS, which corresponds to an approximately 5.7% increase. These results indicate that proactively mitigating the Latency Cliff not only stabilizes execution tail latency but also enhances the overall transaction processing capacity of the system.

4.5 Sensitivity Analysis: Determining the Optimal Spilling Threshold

The performance of LASS heavily relies on the configuration of the memory occupancy threshold (θ), which dictates when the Governor triggers the preemptive state spilling (Step 2). If θ is set too low, the system may suffer from unnecessary I/O overhead. Conversely, if θ is set too high, the spilling mechanism may trigger too late to prevent a massive STW pause. To validate our chosen configuration, we conducted a sensitivity analysis by varying θ at 60%, 75%, and 90% within the JVM (Besu) environment.

Table 1 summarizes the trade-offs between I/O serialization overhead, GC pause times, and overall throughput across different threshold levels.

Table 1 Impact of varying the LASS Governor trigger threshold on system performance.

Threshold (θ)	Avg. Throughput	Jitter (CV)	Latency Cliff Prevention
60% (Aggressive)	97.6 tx/s	85.4%	100%
75% (Optimal)	113.8 tx/s	88.1%	100%
90% (Conservative)	115.2 tx/s	91.5%	70%

Jitter (CV) is defined as the coefficient of variation (i.e., the standard deviation divided by the mean) of per-block execution latency, capturing short-term variability during normal execution rather than rare but catastrophic tail events. As the spilling threshold (θ)

increases, state spilling is triggered later and the materialization of the ephemeral state layer becomes more bursty. However, CV alone does not capture the presence of the Latency Cliff, which is characterized by infrequent but severe Stop-The-World (STW) pauses (A Latency Cliff is defined as a STW pause exceeding 500 ms). In particular, although the conservative configuration ($\theta = 90\%$) achieves the highest raw throughput, it also exhibits a higher CV and a non-negligible probability of Latency Cliff occurrences. In contrast, the $\theta = 75\%$ configuration completely eliminates catastrophic tail latency events while preserving high throughput, making it the most suitable operating point for consensus-driven blockchain systems where worst-case predictability is more critical than average jitter.

- Aggressive Spilling ($\theta = 60\%$): At a lower threshold, the Governor intervenes very early. While this completely eliminates any risk of reaching the Latency Cliff (reducing P99 STW pauses to near zero), it forces the system to materialize and serialize the Ephemeral State Layer too frequently. This aggressive behavior introduces noticeable I/O overhead, slightly degrading the overall throughput.
- Optimal Balance ($\theta = 75\%$): Our default configuration strikes the optimal balance. At 75% occupancy, the Ephemeral State Layer is allowed to absorb a sufficient number of transient objects without prematurely triggering I/O operations. Meanwhile, it leaves enough headroom (25%) in the Old Generation to safely complete background spilling process before the allocation rate (R_a) can trigger a catastrophic GC pause. This configuration achieves near-maximum throughput while fully preventing the Latency Cliff.
- Conservative Spilling ($\theta = 90\%$): When the threshold is set too high, the Governor acts too conservatively. Because the state accumulation is highly bursty (R_a is extreme), the remaining 10% of heap capacity is often consumed before the state spilling process can fully materialize and flush the objects to the persistent backend. Consequently, the runtime occasionally falls back on STW GC, reintroducing severe tail latency spikes.

In conclusion, the sensitivity analysis justifies our selection of $\theta = 75\%$ (the JVM trigger condition in Section 3.3) as the optimal configuration, effectively maximizing transaction throughput while providing a robust safety margin against the Latency Cliff. All experiments were conducted on a single evaluation platform. However, the Latency Cliff is primarily caused by allocation bursts and GC interaction, which are runtime behaviors largely independent of specific hardware characteristics.

5. Discussion

5.1 Cross-Runtime Synergy and Generality

The experimental results across two distinct managed runtimes—JVM and CLR—demonstrate the architectural generality of LASS. While the underlying garbage collection algorithms differ (e.g., G1GC’s generational promotion in the JVM versus the Large Object Heap (LOH) management in the CLR), the fundamental bottleneck remains consistent: the accumulation of short-lived yet complex cryptographic state objects generated during blockchain transaction processing. In the CLR runtime, objects larger than approximately 85 KB are allocated on the Large Object Heap (LOH) rather than the Small Object Heap (SOH), according to the design of the .NET garbage collector. This threshold exists to avoid the high copying cost associated with compacting large objects. Unlike objects in the SOH, LOH objects are generally not compacted during regular garbage collection cycles and are typically reclaimed only during full GC events. Consequently, bursts of large allocations may increase memory fragmentation and trigger longer stop-the-world pauses. In blockchain clients such as Nethermind, transient objects generated during state-trie updates, serialization of account and state nodes, and cryptographic hashing may exceed this threshold. These allocations can therefore be placed on the LOH, amplifying GC pause behavior under heavy allocation pressure. The performance impact of LASS is more pronounced in the JVM environment due to the strong influence of generational promotion pressure on GC pause behavior. In contrast, the CLR runtime tends to exhibit relatively more stable allocation patterns, which explains the smaller but still measurable performance improvements observed in Nethermind. Our findings suggest that the core principle of proactive state spilling is largely runtime-agnostic, although the triggering signals must be adapted to the memory-management characteristics of each runtime. By decoupling the lifecycle of state objects from the generic heuristics of a garbage collector, LASS introduces a specialized memory-management layer tailored for blockchain workloads. Nevertheless, it is important to delineate the boundaries of this generality. While the architectural placement of the LASS governor is universal, the specific trigger signals for state spilling remain runtime-dependent. In our implementations, LASS monitors environment-specific indicators such as 75% Old Generation occupancy in the JVM and the 85,000-byte LOH allocation boundary in the CLR. Furthermore, LASS is explicitly designed to mitigate garbage-collection pressure in managed runtimes. Therefore, its direct application to blockchain clients implemented with manual memory management (e.g., Rust or C++) is limited, although the underlying con-

cept of proactive state spilling may still inspire custom memory allocation strategies in those ecosystems. It is also important to clarify the scope of the evaluated clients. Our experiments focus on managed-runtime blockchain clients (JVM and CLR), as the Latency Cliff phenomenon arises from the interaction between blockchain state workloads and automatic garbage collection. Clients implemented in unmanaged languages, such as Go or Rust (e.g., Geth or Reth), rely on manual memory management and therefore do not experience GC-driven pause amplification in the same manner. Consequently, they are not directly comparable targets for evaluating the effectiveness of LASS, which specifically addresses latency instability induced by garbage collection. The more pronounced improvement observed in the JVM environment reflects the higher sensitivity of generational collectors to promotion pressure. In contrast, the CLR runtime primarily experiences allocation fragmentation within the LOH, resulting in more moderate but still measurable performance improvements. It is important to note that conventional garbage collection tuning alone cannot fully address the Latency Cliff phenomenon. While parameters such as heap size, GC thresholds, or collection frequency may alleviate temporary memory pressure, they do not fundamentally change the allocation behavior of blockchain state workloads. Large bursts of transient state objects can still accumulate and trigger expensive promotion or LOH allocation events. In contrast, LASS proactively restructures the lifecycle of state objects before they exert pressure on the garbage collector. As a result, the mechanism addresses the root cause of latency instability rather than merely adjusting GC heuristics.

5.2 The Predictability-Throughput Trade-off

A key insight from our evaluation is the relationship between I/O overhead and execution predictability. As observed in our sensitivity analysis, aggressive spilling can lead to a reduction in raw throughput due to serialization costs. However, in the context of blockchain consensus, predictability is often more critical than maximum throughput. A validator that maintains a stable response time is less likely to miss block-proposing deadlines than one with higher average speed but occasional 500ms GC pauses. LASS allows node operators to navigate this Pareto frontier by tuning the spilling threshold (θ) to prioritize deterministic execution. While the current static configuration of θ provides a reliable baseline, adapting to highly volatile network conditions necessitates a more dynamic approach, which we discuss further in Section 6.

5.3 Forward Compatibility with Verkle Tries

The forthcoming transition of the Ethereum state ar-

chitecture to Verkle Tries will not render the LASS methodology obsolete. Because future Verkle implementations will continue to rely on in-memory Accumulators to buffer state transitions prior to final commitment, the LASS governor’s design is expected to remain largely structure-agnostic, since it operates on the lifecycle of pre-commit state diffs rather than on the internal structure of the underlying authenticated data structure. By explicitly targeting the pre-commit state diffs that drive heap instability, LASS maintains its efficacy regardless of whether the underlying cryptographic commitment layer utilizes a Modified Merkle Patricia Trie (MPT) or a Verkle structure. This architectural decoupling ensures that LASS serves as a future-proof optimization for the next generation of decentralized networks.

Nevertheless, we acknowledge that our current forward-compatibility claims are primarily based on structural deduction. Because actual Verkle Trie implementations may introduce wider nodes and shallower depths, these fundamental structural shifts may alter serialization costs and GC threshold frequencies in unforeseen ways. Therefore, empirical validation—specifically, benchmarking LASS within a Verkle-enabled testnet to quantify its precise impact on GC pressure and memory access patterns—remains an essential future endeavor to definitively substantiate these claims.

6. Limitations and Future Work

While LASS effectively neutralizes the Latency Cliff and establishes a predictable execution environment, our current evaluation highlights specific constraints that warrant further investigation.

6.1 Evaluation Scope and Workload Realism

Our evaluation primarily uses a synthetic workload designed to induce controlled and reproducible allocation pressure within the managed runtime. This design allows us to isolate the effect of garbage collection behavior and directly compare the Baseline and LASS configurations under identical conditions. However, real-world blockchain workloads exhibit significantly greater diversity, including heterogeneous smart contract executions, token transfers, decentralized finance (DeFi) interactions, and varying storage access patterns. These heterogeneous execution paths may influence object allocation characteristics and garbage collection dynamics in ways that differ from the controlled workload used in this study.

Therefore, while the synthetic workload provides a useful environment for investigating GC-induced latency instability, it does not fully represent the complexity of production blockchain networks. Extending the evaluation of LASS to heterogeneous transaction

mixes and mainnet trace replay remains an important direction for future work. Such experiments would provide further insight into the robustness of the proposed approach under realistic transaction distributions and network conditions.

6.2 Mitigating I/O Contention and Serialization Overhead

Our sensitivity analysis revealed a critical trade-off under aggressive preemptive spilling configurations. When the spilling threshold (θ) was tightly constrained to 60%, the system exhibited extreme latency stability but suffered a 14.2% degradation in overall transaction throughput compared to the optimal 75% setting. This performance penalty primarily stems from the synchronous serialization of state objects; frequent preemptive spilling inadvertently shifts the system bottleneck from GC memory pressure to storage I/O contention, causing a severe I/O amplification effect.

To resolve this, future iterations of LASS will explore asynchronous spilling architectures utilizing dedicated background I/O threads pools and state-diff batching. This approach will effectively decouple the main transaction threads from the I/O-intensive spilling operations. Furthermore, integrating the LASS governor with advanced, write-optimized storage layouts—such as Log-Structured Merge (LSM) trees—could substantially mask the serialization overhead. By minimizing I/O amplification, this integration will ensure that proactive memory offloading does not compromise peak transactional throughput, thereby fully realizing the potential of the LASS architecture. While our evaluation demonstrates that the overall performance impact of LASS is favorable, the current experiments do not isolate the exact per-spill latency cost. Future work will incorporate fine-grained instrumentation to measure serialization time, backend write latency, and spill frequency, enabling a more detailed decomposition of the overhead introduced by LASS state spilling. Preliminary profiling suggests that the dominant component of the spilling overhead arises from serialization rather than disk write latency, as the NVMe backend provides sufficient throughput for the observed workload. Nevertheless, a more systematic breakdown of spill costs remains an important direction for future investigation.

6.3 Dynamic Tuning of Spilling Thresholds

Currently, LASS relies on a statically configured optimal threshold ($\theta = 75\%$) to trigger state spilling. While this static configuration provided a reliable baseline in our controlled benchmarks, it may lack the agility required for highly volatile production environments. As demonstrated in our evaluation, employing a more conservative threshold of 90% failed to provide sufficient

buffer space for the managed runtime, resulting in a 30% recurrence rate of the Latency Cliff. To build resilience against unpredictable bursts in network traffic and varying state mutation rates, our future work will focus on developing an automated, dynamic tuning mechanism. By implementing a feedback-loop system (e.g., a PID controller) that continuously monitors real-time object allocation (R_a) and reclamation (R_r) rates, the LASS governor could autonomously adjust θ to maintain optimal heap stability without requiring manual operator intervention.

7. Conclusion

This paper identified and addressed the Latency Cliff, a systemic execution instability observed in managed blockchain clients caused by the interaction between state-trie workloads and reactive garbage collection behavior. To mitigate this issue, we introduced Lifecycle-Aware State Spilling (LASS), a proactive mechanism that manages the lifecycle of ephemeral state objects before they trigger large garbage collection events.

Our cross-runtime evaluation demonstrated that LASS improves execution stability without requiring modifications to the underlying runtime. In the JVM-based Hyperledger Besu client, LASS significantly reduced promotion pressure, decreasing total GC time by 96.2% and reducing P99 Stop-The-World (STW) pauses by 35.4%, while largely eliminating extreme latency spikes. In the CLR-based Nethermind client, proactive stabilization of Large Object Heap (LOH) allocation reduced GC overhead and improved overall throughput by approximately 5.7%.

These results suggest that proactive lifecycle management of transient state objects can substantially improve execution predictability in managed blockchain clients. Because LASS operates on pre-commit state diffs rather than on the internal structure of the underlying authenticated data structure, the approach is expected to remain applicable to future state architectures, including Verkle Tries.

Overall, this work demonstrates that runtime-aware state lifecycle management can provide an effective systems-level approach for stabilizing execution behavior in high-performance blockchain validators.

References

- [1] V. Buterin, "A next-generation smart contract and decentralized application platform," *Ethereum White Paper*, vol. 3, no. 37, 2014.
- [2] S. Ponnappalli, A. Shah, A. Tai, S. Banerjee, V. Chidambaram, D. Malkhi, and M. Wei, "Scalable and efficient data authentication for decentralized systems," *arXiv:1909.11590*, 2019.
- [3] C. Zhang, C. Xu, J. Xu, Y. Tang, and B. Choi, "GEM²-Tree: A gas-efficient structure for authenticated range queries in blockchain," *Proc. IEEE ICDE*, 2019.
- [4] Z. Chen, B. Li, X. Cai, Z. Jia, Y. Wang, Z. Shao, and Z. Shen, "Block-LSM: An ether-aware block-ordered LSM-tree based key-value storage engine," *Proc. IEEE ICCD*, 2021.
- [5] S. Ponnappalli, A. Shah, S. Banerjee, D. Malkhi, A. Tai, V. Chidambaram, and M. Wei, "RainBlock: Faster transaction processing in public blockchains," *Proc. USENIX ATC*, 2021.
- [6] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A secure sharding protocol for open blockchains," *Proc. ACM CCS*, 2016.
- [7] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "OmniLedger: A secure, scale-out, decentralized ledger via sharding," *Proc. IEEE S&P*, 2018.
- [8] M. Dai, S. Zhang, H. Wang, and S. Jin, "A low storage room requirement framework for distributed ledger in blockchain," *IEEE Access*, vol. 6, pp. 22970-22975, 2018.
- [9] X. Dai, J. Xiao, W. Yang, C. Wang, and H. Jin, "Jidar: A jigsaw-like data reduction approach without trust assumptions for bitcoin system," *Proc. IEEE ICDSC*, 2019.
- [10] D. Detlefs, C. Flood, S. Heller, and T. Printezis, "Garbage-first garbage collection," *Proc. ACM ISMM*, pp. 37-48, 2004.
- [11] G. Tene, B. Iyengar, and M. Wolf, "C4: The continuously concurrent compacting collector," *Proc. ACM ISMM*, pp. 79-88, 2011.
- [12] A. Maoni and D. Detlefs, "Garbage collection for .NET," *Proc. ACM ISMM*, pp. 71-80, 2002.
- [13] E. D. Berger, K. S. McKinley, R. D. Blumofe, and P. R. Wilson, "Hoard: A scalable memory allocator for multithreaded applications," *Proc. ACM ASPLOS*, pp. 117-128, 2000.
- [14] L. Bläser, C. Russo, U. Degenbaev, Ö. Ağacan, G. Greif, and J. Ibrahim, "Collecting garbage on the blockchain," *Proc. ACM VMIL*, pp. 50-60, 2023.
- [15] K. Nguyen, K. Wang, Y. Bu, L. Fang, J. Hu, and G. Xu, "Facade: A compiler and runtime for (almost) object-bounded big data applications," *Proc. ACM ASPLOS*, pp. 675-690, 2015.
- [16] K. Nguyen, L. Fang, G. Xu, B. Demsky, S. Lu, S. Alamian, and O. Mutlu, "Yak: A high-performance big-data-friendly garbage collector," *Proc. USENIX OSDI*, pp. 349-365, 2016.
- [17] J. Bae, H. Jang, W. Jin, J. Heo, J. Jang, J.-Y. Hwang, S. Cho, and J. W. Lee, "Jointly optimizing task granularity and concurrency for in-memory mapreduce frameworks," *Proc. IEEE BigData*, pp. 130-140, 2017.
- [18] H. Jang, J. Bae, T. J. Ham, and J. W. Lee, "Eager memory management for in-memory data analytics," *IEICE Trans. Inf. & Syst.*, vol. E102-D, no. 3, pp. 632-636, 2019.



Yeochan Yoon received B.S. and M.S. degrees from the Department of Electronic Engineering, Sogang University, Seoul, South Korea, in 2007 and 2009, respectively. He is currently pursuing the Ph.D. degree in electronic engineering with Sogang University. His research interests include HW/SW co-design, and high-performance computing.



Yangmo Yoo received B.S. and M.S. degrees from the Department of Electronic Engineering, Sogang University, Seoul, South Korea, in 1999 and 2001, respectively, and the Ph.D. degree from the Department of Bioengineering, University of Washington, Seattle, WA, USA, in 2007. From 2007 to 2009, he was a Systems Design Engineer with Philips Healthcare, Bothell, WA, USA. He is currently a Professor of electronic engineering

at Sogang University. His research interests include high-performance computing, and medical ultrasound imaging.